

Caucasia POS

Cronograma de desarrollo Frontend
Angular 16 + Prompts para Cursor / IA

Inicio sugerido: 30/06/2026

Duracion estimada: 10-12 semanas (1 desarrollador)

Referencia CRUD: features/admin/categorias/

Proyecto: restaurante_frontend | API: <http://127.0.0.1:8000/api>

1. Cronograma de desarrollo

Sprint	Nombre	Inicio	Fin	Entregables
Sprint 0	Fundacion	30/06/2026	06/07/2026	Core, Auth, Shared UI, Categorias, rutas base
Sprint 1	Datos maestros	07/07/2026	20/07/2026	Usuarios + Clientes (CRUD admin)
Sprint 2	Inventario catalogo	21/07/2026	03/08/2026	Insumos + Productos + Recetas
Sprint 3	Caja & POS	04/08/2026	24/08/2026	Turnos, Punto de venta, Dashboard cajero
Sprint 4	Ops & reportes	25/08/2026	07/09/2026	Movimientos, Auditoria, Dashboard admin
Sprint 5	Pulido	08/09/2026	21/09/2026	UX, errores, responsive, tests E2E

Nota: Sprint 0 esta casi completo (~90%). El camino critico es Sprint 2 -> Sprint 3 (POS). Ajusta las fechas segun tu disponibilidad real.

2. Dependencias entre modulos

- Categorias (listo) -> Productos -> Recetas -> POS
- Insumos -> Recetas + Movimientos inventario
- Productos + Arqueo activo -> Punto de venta
- Clientes (opcional) -> Ventas
- Ventas + Arqueos -> Dashboard admin/cajero

3. Mapa de rutas Angular

Ruta	Descripcion	Sprint
/auth/login	Login	Listo
/admin/categorias	CRUD categorias	Listo
/admin/usuarios	CRUD usuarios	Sprint 1
/admin/clientes	CRUD clientes	Sprint 1
/admin/insumos	CRUD insumos	Sprint 2
/admin/productos	CRUD productos + imagen	Sprint 2
/admin/recetas	Recetas producto-insumo	Sprint 2
/admin/turnos	Historial arqueos	Sprint 3
/cajero/turno	Abrir/cerrar turno	Sprint 3
/cajero/pos	Punto de venta	Sprint 3
/admin/inventario/movimientos	Historial stock	Sprint 4
/admin/auditoria	Trazabilidad	Sprint 4

4. Endpoints backend por sprint

Sp	Endpoint	Service	Estado
S0	POST /api/auth/login	AuthService	Listo
S0	GET /api/categorias	CategoriaService	Listo
S1	GET /api/roles	RoIService	Pendiente
S1	GET/POST /api/usuarios/*	UsuarioService	Pendiente
S1	GET/POST /api/clientes/*	ClienteService	Pendiente
S2	GET/POST /api/insumos/*	InsumoService	Pendiente
S2	GET/POST /api/productos/*	ProductoService	Pendiente
S2	GET/POST /api/recetas-producto/*	RecetaService	Pendiente
S3	POST /api/arqueos/abrir	ArqueoService	Pendiente
S3	GET /api/arqueos/activo	ArqueoService	Pendiente
S3	GET /api/metodos-pago	PagoService	Pendiente
S3	POST /api/ventas/crear	VentaService	Pendiente
S4	GET /api/movimientos-inventario	MovimientoService	Pendiente
S4	GET /api/auditoria	AuditoriaService	Pendiente

5. Prompts listos para Cursor / IA

Copia y pega cada prompt en Cursor (modo Agent). Ajusta detalles si el backend cambia. Siempre indica que siga el patron de `features/admin/categorias/`.

PROMPT S0 — Extender rutas y navegacion

En el proyecto Angular `restaurante_frontend`, extiende `core/routing/route-paths.ts` con segmentos y rutas para: usuarios, clientes, productos, insumos, recetas, turnos, auditoria, pos y turno cajero. Actualiza `shared/models/nav.model.ts` para que cada item del sidebar `ADMIN_NAV` y `CAJERO_NAV` apunte a su ruta real. Agrega rutas placeholder en `admin.routes.ts` y `home.routes.ts` con componentes vacios minimo "En construccion". No rompas categorias ni auth. Sigue convenciones existentes.

PROMPT S0 — Servicio de notificaciones toast

Crea un `NotificationService` en `restaurante_frontend/src/app/core/services/` con metodos `success`, `error` e `info`. Integralo en `CategoriaListComponent` para mostrar mensajes al crear o cambiar estado. Usa un componente toast simple en `shared/` o alertas nativas si prefieres minimo scope. Estilo coherente con el layout existente.

PROMPT S1 — Modulo Usuarios completo

Implementa el modulo `admin/usuarios` en Angular 16 siguiendo exactamente el patron de `features/admin/categorias/`. Endpoints: `GET /api/usuarios`, `GET /api/roles`, `POST /api/usuarios/crear`, `POST /api/usuarios/{id}/actualizar`, `POST /api/usuarios/{id}/cambiar_estado`. Crea model, service, list component con tabla busqueda paginador, modal crear/editar con selector de rol, confirmacion cambiar estado. Registra ruta `/admin/usuarios`, `route-paths.ts` y nav. Guard admin. Usa `ApiService`, `ButtonComponent`, `InputComponent`, `ConfirmDialogService`.

PROMPT S1 — Modulo Clientes completo

Implementa `admin/clientes` en `restaurante_frontend` siguiendo el patron de `categorias`. Endpoints: `GET /api/clientes`, `POST /api/clientes/crear`, `POST /api/clientes/{id}/actualizar`, `POST /api/clientes/{id}/estado` (o equivalente). Campos: `tipo_usuario`, `tipo_documento` (FK codigo), `numero_documento`, `nombre`, `apellido`, `correo`, `telefono`, `direccion`, `ciudad`, `departamento`, `pais`, `estado`. Formulario con selector tipo documento. Ruta `/admin/clientes`. Manejo de errores HTTP como en `CategoriaService`.

PROMPT S2 — Modulo Insumos

Crea `features/admin/insumos` en Angular 16. Endpoints `GET /api/insumos` y `POST /api/insumos/crear`. Model con: `id_insumo`, `nombre`, `unidad_medida` (GRAMO|UNIDAD|MILILITRO|LITRO), `stock_actual`, `stock_minimo`, `costo_unitario`, `estado`. List + form modal. Badge de stock bajo si `stock_actual <= stock_minimo`. Patron `categorias`. Ruta `/admin/insumos`.

PROMPT S2 — Modulo Productos con imagen

Implementa `admin/productos`. Endpoints: `GET /api/productos`, `POST /api/productos/crear`, `POST /api/productos/{id}/actualizar`. Campos: `id_categoria` (selector desde `CategoriaService`), `nombre`, `descripcion`, `precio_venta`, `tipo_producto` (PREPARADO|VENTA_DIRECTA), `imagen` (file upload multipart), `estado`. Muestra thumbnail en tabla si hay imagen. Ruta `/admin/productos`. Reutiliza estilos de `categorias`.

PROMPT S2 — Modulo Recetas

Implementa `admin/recetas`. Endpoints `GET /api/recetas-producto` y `POST /api/recetas-producto/crear`. Formulario: selector producto (solo PREPARADO), selector insumo, `cantidad_usada` decimal. Validar duplicado producto+insumo en UI. Tabla muestra nombre producto e insumo. Ruta `/admin/recetas`.

PROMPT S3 — Turnos de caja (cajero + admin)

Implementa flujo de arqueos. Service `ArqueoService`: `POST /api/arqueos/abrir`, `GET /api/arqueos/activo`, `POST /api/arqueos/{id}/cerrar`, `GET /api/arqueos`. Pantalla `/cajero/turno`: si no hay turno, form abrir con `monto_apertura`; si hay turno activo, mostrar resumen y boton cerrar con montos cierre. Admin `/admin/turnos`: tabla historico. Indicador turno activo en layout cajero. Bloquear POS sin turno.

PROMPT S3 — Punto de venta (POS)

Crea /cajero/pos en Angular 16. Requiere turno abierto (GET /api/arqueos/activo). Grid productos activos agrupados por categoria (GET /api/productos). Carrito lateral: cantidad, precio_unitario, subtotal, total. Cliente opcional (GET /api/clientes). Metodos pago desde GET /api/metodos-pago. Campo referencia si aplica. Confirmar con POST /api/ventas/crear enviando id_arqueo, id_cliente opcional, detalles y pagos. UX rapida para tablet. Estilo POS moderno.

PROMPT S3 — Dashboard cajero con datos reales

Reemplaza datos mock en HomeDashboardComponent. Muestra: turno activo (ArqueoService), accesos a /cajero/pos y /cajero/turno, resumen ventas del turno si el backend lo permite o placeholder con mensaje. Elimina constantes mock. Loading y error states.

PROMPT S4 — Movimientos de inventario

Crea /admin/inventario/movimientos. Solo lectura. GET /api/movimientos-inventario. Tabla: insumo, tipo_movimiento, cantidad, motivo, usuario, fecha. Filtros por tipo y busqueda. Paginacion client-side inicial. Ruta y nav admin.

PROMPT S4 — Auditoria

Implementa /admin/auditoria. GET /api/auditoria. Tabla: fecha, usuario, modulo, accion, tabla_afectada, id_registro. Filtro por modulo y rango fechas si aplica. Solo lectura. Estilo tabla admin existente.

PROMPT S4 — Dashboard admin KPIs

Actualiza AdminDashboardComponent reemplazando KPIs mock. Muestra tarjetas: categorias activas, productos activos, turno abierto si existe, ultimos movimientos o auditoria. Usa servicios existentes. Si falta endpoint agregado, muestra dato parcial con TODO comentado.

PROMPT S5 — Pulido y permisos nav

Filtra ADMIN_NAV y CAJERO_NAV segun rol del usuario logueado. Admin ve todo admin; cajero solo rutas cajero. Interceptor: en 401 limpiar token y redirigir login. Loading spinner en listas. Revisa responsive del POS en 768px.

PROMPT GENERICO — Nuevo modulo CRUD

Crea el modulo admin/{MODULO} en restaurante_frontend Angular 16 copiando el patron exacto de features/admin/categorias/: models, service con ApiService, list component, modal form, ruta lazy en admin.routes.ts, entrada en route-paths.ts y nav.model.ts. Endpoints: {LISTAR}, {CREAR}, {ACTUALIZAR}. Manejo errores como CategoriaService.parseError. Guard admin. SCSS coherente con categorias.

6. Checklist por modulo

- [] Model TypeScript alineado al API
- [] Service con listar, crear, actualizar, cambiarEstado
- [] ListComponent: tabla, busqueda, paginador
- [] Modal o formulario crear/editar
- [] ConfirmDialog antes de cambiar estado
- [] Ruta en admin.routes.ts o home.routes.ts
- [] Entrada en route-paths.ts
- [] Nav sidebar actualizado
- [] Guard de rol aplicado
- [] Probado contra `http://127.0.0.1:8000/api`
- [] Mensajes de error visibles (toast o alert)

7. Estructura de carpetas por modulo

```
features/admin/{modulo}/  
  models/{modulo}.model.ts  
  services/{modulo}.service.ts  
  {modulo}-list/  
    {modulo}-list.component.ts  
    {modulo}-list.component.html  
    {modulo}-list.component.scss  
  
core/routing/route-paths.ts  <- ROUTE_SEGMENTS + ROUTES  
shared/models/nav.model.ts   <- ADMIN_NAV / CAJERO_NAV  
features/admin/admin.routes.ts <- lazy load
```

8. Roles UI

- Admin (id_rol=1): /admin/* completo
- Cajero: /cajero, /cajero/turno, /cajero/pos unicamente
- POS bloqueado sin arqueo ABIERTO
- Referencia guards: core/auth/guards/